**#1 Instantiates the kernel**
**#2 Imports the plugin service**
**#3 Inputs parameter to functions, when needed**
**#4 Executes and tests the various functions**
**#5 Inputs parameter to functions, when needed**
**#6 Executes and tests the various functions**
**#7 Inputs parameter to functions, when needed**
**#8 Executes and tests the various functions**
**#9 Executes and tests the various functions**
**#10 Executes main asynchronously**
**#11 Calls print function details to notify when the function is being called**

The real power of SK is shown in this test. Notice how the `TMDbService` class is imported as a plugin, but we don't have to define any plugin configurations other than what we already did? By just writing one class that wrapped a few API functions, we've exposed part of the TMDB API semantically. Now, with the functions exposed, we can look at how they can be used as plugins for a chat interface in the next section.

### 5.5.3 Interactive chat with the semantic service layer

With the TMDB functions exposed semantically, we can move on to integrating them into a chat interface. This will allow us to converse naturally in this interface to get various information, such as current top movies.

Open `SK_service_chat.py` in VS Code. Scroll down to the start of the new section of code that creates the functions, as shown in listing 5.21. The functions created here are now exposed as plugins, except we filter out the chat function, which we don't want to expose as a plugin. The chat function here allows the user to converse directly with the LLM and shouldn't be a plugin.